

Resource Efficiency in Software: A Comparative Study of Two Text Editors

Lena Fischer ¹, Marian Messer ², Dennis Junger ³, and Volker Wohlgemuth⁴

Abstract: The following report documents a practical measurement to compare the two text editors Visual Studio Code (VS Code) and Pulsar. Due to the ongoing increase in available software products for a developer's toolkit, this study aims to compare the energy and hardware consumption to give developers a guideline which text editor to choose when considering energy and resource efficiency. In the used measurement setup, based on the requirements for the Blue Angel for Software, VS Code proves to be the more resource-efficient text editor, particularly in terms of CPU, RAM and hard disk usage. A key finding of this analysis is that VS Code, a software developed by a large corporation, is not more resource-intensive than Pulsar, a community-led open source project. The results indicate that VS Code is significantly more efficient in terms of CPU usage, memory load, and hard disk activity, both in absolute terms and after adjusting for system baseline consumption. Pulsar consumes more energy and system resources in the proposed usage scenario. A critical finding—observed during this and a second comparable study—concerns the use of Power Automate: energy measurements show that the automation tool itself may consume more power during idle phases than the editors during active usage. Hence, this study recommends alternative scripting tools for future experiments and careful validation of the baseline setup to avoid a distorted set of results due to the used measurement methodology. The findings point to the need for improved measurement setups in future studies, including longer measurement periods, more complex usage scenarios, and more efficient automation solutions.

Ultimately, the study aims to contribute to the field of sustainable software by delivering reproducible data and encouraging critical discussion about resource efficiency in everyday programming tools.

Keywords: Ressource Efficiency; Green IT; Open Source Software; Energy Consumption; Text Editor; Sustainability of Software; Green Coding

1 Thematic Introduction and Theoretical Background

The tension between economy and ecology is a well-known concept. In the software industry in particular, experience shows that ecology often loses out to economics. New software is entering the market, computers are becoming more and more power-intensive and data centers are already causing the same amount of CO₂ equivalents as global air traffic [As21]

¹ HTW Berlin, Environmental Informatics Unit, Wilhelminenhofstr. 75A, 12459 Berlin, Germany, lena.fischer@student.htw-berlin.de,  <https://orcid.org/0009-0001-8030-624X>

² HTW Berlin, Fachbereich 2, Wilhelminenhofstraße 75A, 12459 Berlin, Deutschland, marian.messer@student.htw-berlin.de,  <https://orcid.org/0009-0009-9861-5401>

³ HTW Berlin, Fachbereich 2, Wilhelminenhofstraße 75A, 12459 Berlin, Deutschland, dennis.junger@htw-berlin.de,  <https://orcid.org/0000-0002-9271-2261>

⁴ HTW Berlin, Fachbereich 2, Wilhelminenhofstraße 75A, 12459 Berlin, Deutschland, volker.wohlgemuth@htw-berlin.de

in 2021. This topic is addressed by the subject area of green IT, which was explored in greater depth as part of a newly developed module at HTW Berlin [JW23]. The presented measurement setup, based on the methodology of [JW22], is intended to build on this idea and provide information about sustainable, green IT practices through the detailed analysis of the energy consumption and hardware utilization [Ju23], which has been growing steadily for years, with data and its interpretation. Since text editors are part of the daily toolbox this study aims to offer transparency on a software's energy and resource efficiency, giving developers the opportunity to choose sustainable software in their day-to-day work. This report also aims to tackle the topic of open source software and the associated sustainability potential.

1.1 Importance of Text Editors for Software Development

Technologies in software development are subject to constant change. In order to be able to program efficiently and clearly, Integrated Development Environments (IDEs) with a variety of integrated functionalities gained in importance as early as the 1990s and 2000s [Ka24]. While IDEs can still be found in the software landscape of most developers today, another type of coding software has gained importance: lightweight text editors. Text editors such as VS Code, Sublime Text or Notepad++ offer a lean basic framework of functionalities compared to heavyweight IDEs. Required functionalities can then be added as required using extensions and plug-ins.

1.2 Open Source Approach

Open source approaches in software development have been a common practice for decades. While companies often keep software projects internal in order to protect business secrets or gain a competitive advantage, the open source concept illustrates a different approach. The code can be viewed and used by the public. In theory, this approach is intended to increase quality through its practically unrestricted peer review - i.e. the evaluation and improvement of the code by other developers [Ab07]. Since the comparison in this paper deals with two products with opposing approaches, the energy consumption could provide an indication of possible advantages or disadvantages of OSS in the case of lightweight software such as a text editor.

1.3 Frame of Reference

VS Code is one of the most widely used text editors on the market. It is a free software product for both private and commercial use that is developed and maintained by Microsoft. The basic version of VS Code is an intuitive text editor that allows you to open, edit and

save source code files. Furthermore, the ‘out of the box’ version of VS Code includes some standard formatting templates such as the color coding of keywords in the code or automatic indentation of lines. However, the majority of the functionalities offered are not pre-installed during download, but are available via the extension marketplace. Both verified software publishers such as Microsoft and individual users can publish extensions there. These then enable a range of different functionalities, starting with the opening of file types such as PDF files, individualized colour themes and code formatting through to the integration of ChatGPT to support the programming process. For our comparison, the basic version of VS Code without extensions was used due to the comparability with the second software product.

In addition to VS Code, Pulsar was chosen as the comparison product for this practical measurement. This text editor is an evolution of the text editor Atom, which has been retired since 2022 due to security risks [Gi25]. Atom’s popularity in the software development community gave way to the software Pulsar, ‘A Community-led Hyper-Hackable Text Editor’ [Pu25]. It was chosen for this study’s comparison because, unlike VS Code, it is not a software product developed by a corporation like Microsoft that takes over maintenance and further development. Section 1.2 has already raised important arguments in favor of using open source software, which is why Pulsar was chosen as the software to contrast with VS Code. All further developments are open source, the software itself and all updates are free of charge and the Pulsar code is publicly available on GitHub. Similar to VS Code, the download version of Pulsar contains a small number of different standard functionalities such as code highlighting and formatting. Like in VS Code, plugins can be integrated via a marketplace. For a valuable comparison, the ‘atom-terminal’ extension was selected for the integration of a terminal, as this was necessary for the proposed usage scenario. Apart from this, the out-of-the-box version of the software was used here in order to achieve a comparable result.

2 Approach and Methodology

The following sections describe the proposed usage scenario, structure of the measurement setup, and the final evaluation of the collected data using the O.S.C.A.R. software (<https://oscar.umwelt-campus.de/>).

2.1 Use Scenario

In order to keep the measurements of the two text editors as close to reality as possible, a standard usage scenario, consisting of basic interactions with the software, was selected. This was then executed in both VS Code and Pulsar and used for the later measurements. The scenario was then recorded by the automation software *Power Automate* (<https://make.powerautomate.com/>) and repeated 30 times in succession to obtain a representative result. Before the iterations are started, the variables *process time* and *start*

time are created and saved. The following behavior diagram describes the sequence the individual interactions with the two text editors.

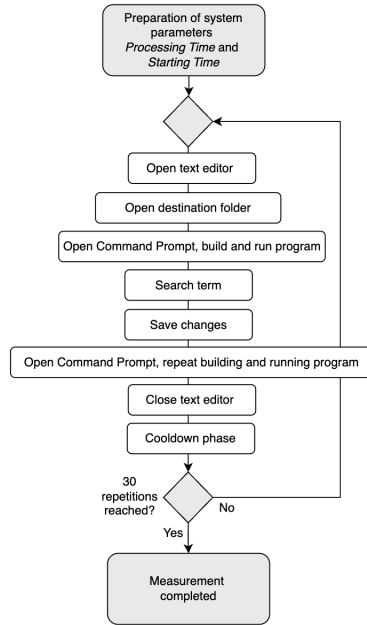


Fig. 1: Behavior diagram for usage scenario (own figure)

2.2 Measuring Setup

In order to obtain representative and comparable measured values, the measurement laboratory provided by HTW Berlin was used. The setup is based on Gröger et. al. [Gr18] and Junger et. al. [JW22]. For the evaluation, the hardware utilization of the System Under Test (SUT) was measured by the built-in Performance Monitor and saved for later processing. The action logs of the usage scenario were also saved. Finally, the electrical power consumption of the SUT ('DJ Darwin') was recorded and saved on a separate PC ('DJ Einstein') using a GUDE Power Control 1202 measuring device while the usage scenario was being played. In addition to the measurements of the usage scenario in both text editors, baseline measurements were recorded that do not contain any user interaction with the software.

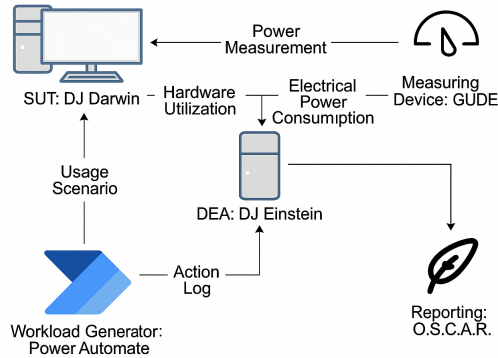


Fig. 2: Measurement setup (own figure, based on Gröger et. al.)

2.3 Evaluation using O.S.C.A.R.

The web version of the O.S.C.A.R. software, developed by the Umweltcampus Birkenfeld, was used to evaluate the recorded measurement data. It uses a defined methodology to generate a report on various factors related to the environmental impact of software. The hardware utilization, electrical power consumption and the action log described in the previous chapter are uploaded as input. This is done for the measurements of the usage scenario and for the baseline measurement. O.S.C.A.R. generates a report, which links the baseline and usage scenario measurements and provides a variety of different results. Due to technical problems with O.S.C.A.R., not all measurement data could be used fully, meaning that the usage scenario for the final evaluation could only be evaluated with ten repetitions instead of the initially planned 30 repetitions.

3 Results and Evaluation

This chapter presents the results of the measurements for the VS Code and Pulsar text editors. Both the absolute values and the baseline-adjusted values are analyzed and finally evaluated.

3.1 Results VS Code

This section describes and analyzes the results of the VS Code measurements in particular.

3.1.1 Measurement Values

The measurements show that the resource utilization of VS Code varies depending on the usage scenario. During the first ten seconds in which the editor is opened, the average processor utilization is 3.78 %, while the memory consumption is 54.77 %. Network usage shows 2645.75 bytes of data sent and 3453.31 bytes of data received. The hard disk activity is 10,259.68 bytes for read accesses and 130,904.8 bytes for write accesses (see O.S.C.A.R. - report VS Code [FM25]).

Between the 20th and 30th second, in which the code is compiled and executed, an increase in CPU and hard disk utilization can be observed. During the thirtieth to thirty-fifth second, when a text search and replace is performed, CPU and RAM remain almost constant. The greatest hard disk activity occurs between the thirty-fifth and fortieth second when the file is saved. The energy consumption measurement for VS Code resulted in an average electrical work of 0.48 Wh in the usage scenario.

3.1.2 Corrected Measurement Values

After subtracting the baseline values, Visual Studio Code shows an additional CPU usage of 0.61%, and an increase in RAM usage of 7.82%. The corrected network traffic amounts to 2402.88 bytes sent and 2209.17 bytes received. Regarding hard disk activity, baseline-adjusted values result in 10,259.68 bytes for read operations and 130,904.80 bytes for write operations.

The corrected energy consumption is calculated at -0.12 Wh. This negative value is not physically plausible and indicates a possible distortion caused by the measurement setup – for example, background load introduced by the automation tool. Nevertheless, this value still reflects the relatively low energy consumption of VS Code within the measured usage scenario. (see O.S.C.A.R. - report VSCode [FM25]).

3.2 Pulsar Results

This chapter describes and analyzes the results of Pulsar's measurements in particular.

3.2.1 Measurement Values

The CPU utilization of Pulsar is 5.11 % on average, while the memory consumption is 61.50 %. The network usage is 1161.33 bytes sent and 2394.31 bytes received. The hard disk activity of 40,028.18 bytes for read accesses and 360,231.8 bytes for write accesses is

significantly higher than with VS Code. An electrical work of 0.56 Wh was measured for Pulsar in the usage scenario. (see O.S.C.A.R. - Report [FM25]).

The higher CPU and RAM utilization is particularly noticeable when the editor is started. During the compilation process between the twentieth and thirtieth second, the hard disk activity remains significantly higher. The network load remains relatively constant in almost all scenario steps.

3.2.2 Corrected Measurement Values

For Pulsar, the baseline-adjusted CPU usage amounts to 2.01%, and RAM usage increases by 10.74%. The corrected network traffic is 1089.64 bytes sent and 1393.71 bytes received. The disk activity is 40,028.18 bytes for reads and 360,231.80 bytes for writes.

The energy consumption corrected for baseline is -0.03 Wh. As with VS Code, this negative value suggests that the measurement setup introduced inconsistencies, possibly due to overhead from the automation tool. However, when comparing the two editors, Pulsar still consumed approximately 0.08 Wh more energy than VS Code in the same scenario, which is consistent with the observed differences in CPU, RAM, and disk utilization. (see O.S.C.A.R. - report [FM25]).

Metric	VS Code	Pulsar	Difference (Pulsar – VS Code)
CPU usage (%)	0.61	2.01	+1.40
RAM usage (%)	7.82	10.74	+2.92
Network – sent (B)	2402.88	1089.64	–1313.24
Network – received (B)	2209.17	1393.71	–815.46
Disk – read (B)	10,259.68	40,028.18	+29,768.50
Disk – written (B)	130,904.80	360,231.80	+229,327.00

Tab. 1: Comparison of resource usage between VS Code and Pulsar

3.3 Discussion

The comparison shows that VS Code is more resource-efficient overall than Pulsar. The baseline-adjusted values confirm this trend: Pulsar requires more CPU and RAM resources and has higher hard disk activity. VS Code, on the other hand, shows increased network usage, which could indicate online services or synchronization processes.

Another possible technical influencing factor is the underlying architecture of the programs: While VS Code is based on Electron and implemented with TypeScript/JavaScript, Pulsar also uses the Electron platform, but originally stems from the Atom project (also developed in JavaScript), whose code structure is considered less efficient. Differences in garbage collection, thread handling, or rendering can therefore have a direct impact on the measured CPU and RAM utilization.

External background processes during the measurement as well as the selected usage scenario limit the general validity of the results. Further analysis with alternative scenarios and longer test durations would be necessary to derive broader conclusions.

The energy consumption in watt hours (Wh) calculated by O.S.C.A.R. was also used to evaluate the energy efficiency. The corrected values adjusted for the system baseline show slightly negative energy consumption for both text editors (VS Code: -0.12 Wh, Pulsar: -0.03 Wh). While not physically plausible, these values indicate measurement distortions, potentially caused by the automation tool Power Automate or inaccuracies in the baseline reference.

Nevertheless, it is noticeable that Pulsar consumed approximately 0.08 Wh more energy than VS Code, which corresponds with the observed differences in CPU, RAM, and disk usage. This supports the overall finding that VS Code demonstrates higher runtime efficiency within the chosen scenario.

However, it is important to emphasize that energy consumption alone does not determine software sustainability. While runtime efficiency is a relevant metric, sustainability also encompasses long-term maintainability, update frequency, hardware compatibility, licensing, and community engagement. Open-source software like Pulsar can offer significant advantages in these areas for instance, by allowing transparent code inspection, avoiding vendor lock-in, and supporting older systems longer.

Therefore, this study's findings should not be interpreted as a general assessment of proprietary software being more sustainable than open source alternatives. Rather, they highlight that in this specific test case, VS Code exhibited greater energy and resource efficiency. Broader sustainability assessments should include diverse tools, scenarios, and lifecycle phases to ensure a more holistic understanding.

4 Critical Reflection and Additional Findings

The evaluation results revealed that the use of Power Automate as an automation tool significantly influenced the energy consumption measurements, especially during idle phases. In some cases, the baseline consumption while the system was idle but Power Automate was active exceeded the consumption during actual test execution. This behavior introduces distortions in the calculated energy values, as baseline consumption is subtracted from the measured usage scenario.

These findings align with those from a related study [MR25a], [MR25b], which observed similar issues when Power Automate was used to automate energy measurements. One likely reason is that Power Automate introduces system load even when no active input is being performed, possibly due to inefficient polling or active waiting processes.

To improve future experiments and reduce external influences on the measurement data, we recommend using lightweight and script-based automation tools. Tools such as *Shell scripts*, *Actiona*, or *AutoHotkey* could be more resource-efficient and easier to control in terms of background load and processor utilization. In addition, they allow more precise timing and scenario control, which can improve reproducibility.

Further technical improvements could involve pinning processes to specific CPU cores or using real-time process monitoring to exclude side effects caused by system background processes.

5 Conclusion

The measurements carried out show that Visual Studio Code works more resource-efficiently overall than the open source editor Pulsar in the usage scenario examined. This applies both to the utilization of central system components (CPU, RAM, hard disk) and, with restrictions, to the measured energy consumption. Despite possible distortions due to Power Automate, the adjusted consumption values indicate a consistent efficiency advantage of VS Code.

One key takeaway of this study is that software developed by a large corporation is not necessarily more resource intensive than a community-led open source project. However, this observation is limited to runtime efficiency in a controlled usage scenario. Broader sustainability implications such as long-term maintainability, support for older hardware, transparency, and community engagement were not part of this evaluation and should not be inferred from the presented data.

Therefore, this case study does not contradict the potential of open-source software to contribute to sustainable software development. Rather, it highlights the need to assess sustainability based on multiple measurable criteria beyond energy and resource consumption alone.

For a more in depth analysis, the use of alternative automation tools instead of Power Automate is recommended to minimize possible distortions caused by background load. Measurement over longer periods of time or with more complex interaction patterns such as parallel editing, live debugging, or the use of plugins could also provide deeper insights into the resource dynamics of the editors.

Future studies could additionally examine aspects such as update frequency, system compatibility, and energy usage during software installation and maintenance. Including other open-source editors such as Sublime Text or Neovim would also help to contextualize the findings in a broader Green IT discourse.

Acknowledgments

This work was funded by the Internet Society Foundation⁵.

References

- [Ab07] Aberdour, M.: Achieving Quality in Open Source Software. *IEEE Software* 24 (1), pp. 58–64, 2007.
- [As21] Association for Computing Machinery: Computing and Climate Change. *Tech Briefs* 1, 2021.
- [FM25] Fischer, L.; Messer, M.: Zenodo Repository: Green IT: Software Measurements of Text Editors Visual Studio Code and Pulsar, 2025, <https://zenodo.org/records/15050857>, (last accessed on 19.03.2025).
- [Gi25] GitHub Staff: Sunsetting Atom, 2025, <https://github.blog/news-insights/product-news/sunsetting-atom/>, (last accessed on 27.02.2025).
- [Gr18] Gröger, J. et al.: Entwicklung und Anwendung von Bewertungsgrundlagen für ressourceneffiziente Software unter Berücksichtigung bestehender Methodik. Umweltbundesamt, 2018.
- [Ju23] Junger, D. et al.: Potentials of Green Coding -Findings and Recommendations for Industry, Education and Science. *INFORMATIK 2023 - Designing Futures: Zukünfte gestalten*, 2023.
- [JW22] Junger, D.; Wohlgemuth, V.: Conception and test of a measuring station for the analysis of the resource and energy consumption of material flow-oriented environmental management information systems (EMIS). *EnviroInfo* 2022, 2022.
- [JW23] Junger, D.; Wohlgemuth, V.: Design and implementation of a lecture for teaching current Green Coding approaches and practices at the HTW Berlin. *INFORMATIK 2023 - Designing Futures: Zukünfte gestalten*, 2023.
- [Ka24] Kapoor, A.: Analysis of Code Editors: Features, Evolution, and Impact on Software Development. *IJCAM*, 2024.
- [MR25a] Melvana Mauladendra, I.; Rauch, S.: Resource consumption based usability study for the comparison between customer segmentation with KMeans clustering using tensorflow and scikit-learn implementation, 2025, <https://zenodo.org/records/14991212>, (last accessed on 24.06.2025).
- [MR25b] Melvana Mauladendra, I.; Rauch, S.: Resource consumption based usability study for the comparison between customer segmentation with KMeans clustering using tensorflow and scikit-learn implementation, 2025, <https://zenodo.org/records/14991151>, (last accessed on 24.06.2025).
- [Pu25] Pulsar-Edit: Pulsar Homepage, 2025, <https://pulsar-edit.dev/>, (last accessed on 27.02.2025).

⁵ <https://www.isocfoundation.org/> [2025-06-06]